14 haftalık yol haritasında 1. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 01

Bilgisayar Nasıl Çalışır? Programın Yaşam Döngüsü ve İlk C Programı

Soyutlama katmanları, Von Neumann mimarisi, ikili sayılar ve derleme hattı

MODÜL I · MAKİNE VE ALGORİTMA · H1–H2

HAFTANIN BÜYÜK SORUSU

Klavyede yazdığımız printf("Merhaba"); satırı, ekranda piksellere dönüşene kadar hangi katmanlardan geçer?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Büyük Soru ve Öğrenme Çıktıları 3
2. Ön Bilgi Yoklaması 3
3. Kavramsal Anlatım 4
3.1 Soyutlama katmanları 4
3.2 Von Neumann mimarisi 5
3.3 Getir–çöz–yürüt döngüsü ve oyuncak bilgisayar 5
3.4 Bellek hiyerarşisi 7
3.5 Verinin temsili: bit, bayt, ikili ve onaltılı 8
3.6 Makine dilinden C'ye 9
3.7 Derleme hattı 10
3.8 İlk C programının anatomisi 11
3.9 Geliştirme ortamı 12
4. Çözümlü Örnekler: Örnek Merdiveni 13
4.1 Örnek 1 ★ · Merhaba, Dünya! 13
4.2 Örnek 2 ★ · Çok satırlı çıktı ve kaçış dizileri 13
4.3 Örnek 3 ★ · ASCII sanatıyla isim kartı 14
4.4 Örnek 4 ★ · printf ile ilk hesaplama 15
4.5 Örnek 5 ★ · Noktalı virgülü eksik program 16
4.6 Örnek 6 ★★ · Derleyici mi, bağlayıcı mı? (amiral) 16
4.7 Örnek 7 ★★ · Çıkış kodu 19
4.8 Örnek 8 ★★ · gcc -save-temps ile ara dosyalar (amiral) 19
4.9 Örnek 9 ★★★ · -O0 ile -O2 (Merak) 22
5. Hata Avcısı 23
6. Mühendis Gözüyle 24
7. Mini-OBS Projesi: v0.1 24
8. Alıştırmalar 25
9. Kendini Değerlendirme 27
10. AP-2 Köprüsü ve Merak Kutusu 28
11. Özet, Kavram Haritası ve Terim Sözlüğü 28
12. Kaynaklar 30

1. Büyük Soru ve Öğrenme Çıktıları

Bu dersin ilk satırı çoğu zaman `printf("Merhaba, Dünya!\n");` olur. Bu hafta bu tek satırın izini süreceğiz: Karakterler bir dosyaya yazılır, bir dizi programdan geçerek işlemcinin anladığı sayılara dönüşür, bellekte yer alır, işletim sistemine bir istek olarak ulaşır ve sonunda ekranda birkaç yüz pikselin rengini değiştirir. Haftanın Büyük Sorusu: **Bu satır, ekranda piksellere dönüşene kadar hangi katmanlardan geçer?** Yanıtı hafta boyunca parça parça kurup Bölüm 11'de (Şekil 1.12) birleştireceğiz.

> GERÇEK DÜNYADAN • **9 Eylül 1947: Kayıt defterine yapıştırılan güve**
>
> Harvard Üniversitesi'ndeki **Harvard Mark II**, binlerce elektromekanik röleyle çalışan bir hesap makinesiydi. 9 Eylül 1947'de makineyi işleten ekip, bir arızanın nedenini ararken 70 numaralı rölenin kontakları arasına sıkışmış bir güve buldu. Güve kayıt defterine bantla yapıştırıldı ve yanına *"First actual case of bug being found."* (Gerçek bir böceğin bulunduğu ilk vaka) notu düşüldü.
>
> Esprinin kaynağı, sözcüğün zaten kullanılıyor olmasıdır: Thomas Edison 1870'lerde devrelerdeki "bug"lardan söz etmişti. Olay sözcüğü icat etmedi, **simgeleştirdi**: Ekipteki matematikçi ve Donanma subayı **Grace Hopper** öyküyü yıllarca anlatarak yaygınlaştırdı (güveyi bulan kişi Hopper değildi). Defter bugün Smithsonian Ulusal Amerikan Tarihi Müzesi'ndedir (National Museum of American History, t.y.).

> GERÇEK DÜNYADAN • **1952: Makineye kendi programını yazdırmak**
>
> Hopper'ın asıl katkısı başka yerdedir. O yıllarda programlar makinenin sayısal kodlarıyla elle yazılıyordu. Hopper, UNIVAC I için geliştirdiği **A-0 sistemiyle** (1952) bu işi makineye devretti: Programcı kısa simgesel kodlar yazıyor, A-0 gerekli alt programları bir kütüphaneden bulup birleştirerek makine kodunu üretiyordu (Hopper, 1952). A-0 bugünkü anlamda tam bir derleyici değildi; ama "programı insan yazsın, makine koduna çevirmeyi başka bir program yapsın" fikri bu hafta inceleyeceğimiz **derleme hattının** (compilation pipeline) atasıdır.

Öğrenme çıktıları. Bu haftanın sonunda şunları yapabilmelisiniz:

- **ÖÇ 1.1** Bilgisayar sistemini soyutlama katmanları (donanım, komut kümesi mimarisi, işletim sistemi, derleyici, uygulama) üzerinden **açıklar** (Bölüm 3.1).
- **ÖÇ 1.2** Von Neumann mimarisinin bileşenlerini (CPU: ALU, kontrol birimi, yazmaçlar; bellek; G/Ç; veri yolu) ve getir–çöz–yürüt döngüsünü **açıklar** (3.2–3.4).
- **ÖÇ 1.3** Bit, bayt, ikili ve onaltılı sayı sistemlerinde temel **dönüşümleri yapar**; verinin bellekte ikili örüntüler olarak saklandığını kavrar (3.5).
- **ÖÇ 1.4** Kaynak koddan çalışan sürece uzanan derleme hattını (önişlemci → derleyici → çevirici → bağlayıcı → yükleyici) adım adım **açıklar** (3.7, Örnek 8).
- **ÖÇ 1.5** Geliştirme ortamını **kurar**; ilk C programını derler, çalıştırır ve derleyici ile bağlayıcı hata iletilerini **ayırt eder** (3.9, Örnek 5–6, Bölüm 5).

2. Ön Bilgi Yoklaması

Dersin ilk haftası olduğu için geçen haftayı hatırlatmak yerine sezgilerinizi yoklayarak başlıyoruz. Aşağıdaki soruları okuyun ve **devam etmeden önce** kısa yanıtlarınızı yazın. Amaç not almak değil, hafta sonunda sezginizin nasıl değiştiğini görmektir; değerlendirme Bölüm 9'dadır.

TAHMİN ET · Başlamadan önce üç soru

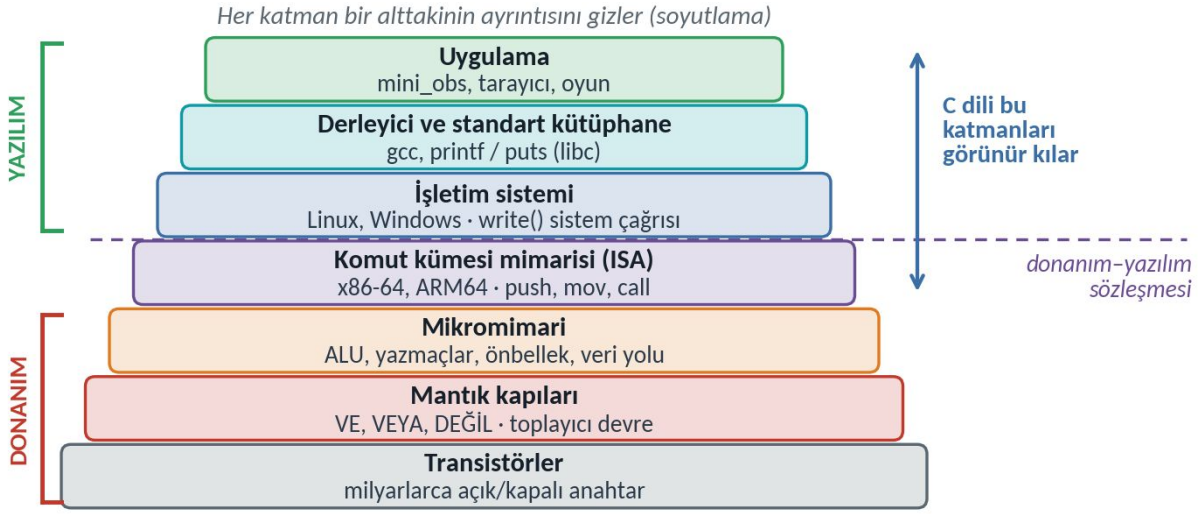
- **Soru 1.** Bilgisayarınızın fişini çektiğinizde açık belgenin RAM'deki kopyasına ne olur? Diskteki kopyasına? Neden iki ayrı yer var? (→ Bölüm 3.4)
- **Soru 2.** Bilgisayar "A" harfini, bir fotoğrafı ve çalışan bir programı aynı bellekte nasıl saklar? Bunların ortak "dili" nedir? (→ Bölüm 3.2, 3.5)
- **Soru 3.** İşlemci yazdığınız C programını doğrudan anlar mı? Anlamıyorsa aradaki "çevirmen" kimdir ve kaç adımda çalışır? (→ Bölüm 3.7)

3. Kavramsal Anlatım

3.1 Soyutlama katmanları

Bir otomobili kullanmak için motordaki yanmayı bilmemiz gerekmez: Pedala basarız, araç hızlanır. Pedal, altındaki karmaşık sistemin **soyutlamasıdır** (abstraction). Bilgisayar da üst üste binmiş soyutlama katmanlarından oluşur; her katman alttakinin ayrıntılarını gizler, üsttekine daha basit bir arayüz sunar (Şekil 1.1). Bu dersin **"Kaputun Altında"** kutuları tam da bu kaputu kaldırıp bir alt katmana bakmak içindir.

Şekil 1.1
Bir bilgisayar sisteminin soyutlama katmanları



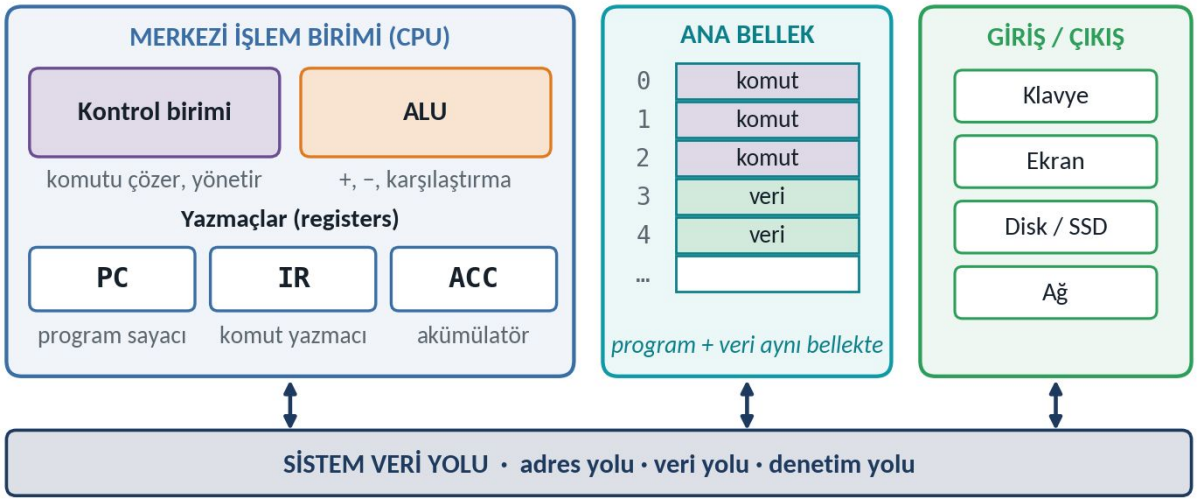
En altta milyarlarca **transistör**, onlardan kurulan **mantık kapıları** (VE, VEYA, DEĞİL) ve kapılardan kurulan **mikromimari** (ALU, yazmaçlar, önbellekler) vardır. **Komut kümesi mimarisi** (ISA, instruction set architecture; ör. x86-64, ARM64) işlemcinin tanıdığı komutların listesidir ve donanım ile yazılım arasındaki **sözleşmedir**: Aynı ISA'yı uygulayan farklı işlemciler aynı makine kodunu çalıştırır. **İşletim sistemi** donanımı programlar arasında paylaştırır ve dosya, süreç, ekran gibi kaynaklara **sistem çağrıları** (system call) üzerinden erişim verir. **Derleyici ve standart kütüphane** C kodunu makine koduna çevirir (`gcc`) ve `printf` gibi hazır fonksiyonları sağlar (libc). En üstte kullanıcının gördüğü **uygulama** durur.

C bu katmanların ortasındadır: Yüksek düzeyli bir dildir ve aynı kaynak kod farklı işlemcilerde derlenebilir; ama belleği, adresi ve veri temsilini gizlemediği için alttaki katmanları görünür kılar. C'ye bu yüzden zaman zaman "taşınabilir assembly" denir. **Kaputun altında:** Katmanları komut satırından sorgulayabiliriz; bu belgedeki bütün çıktıların üretildiği makinede ISA `uname -m` → `x86_64`, işletim sistemi Linux (Ubuntu 24.04, ELF biçimi), derleyici `gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0`'dır.

3.2 Von Neumann mimarisi

Günümüz bilgisayarlarının neredeyse tamamı, John von Neumann'ın 1945'te EDVAC için yazdığı rapordaki düzeni izler (von Neumann, 1945; Şekil 1.2). **CPU** (merkezi işlem birimi) komutları sırayla getirir, çözer ve yürütür; içinde komutu çözüp yöneten **kontrol birimi** (control unit), toplama, çıkarma ve karşılaştırma yapan **ALU** (arithmetic logic unit) ve çok hızlı küçük saklama alanları olan **yazmaçlar** (registers) bulunur: **PC** (program sayacı) sıradaki komutun adresini, **IR** yürütülen komutu tutar. **Ana bellek** (RAM) numaralı hücrelerden oluşur; her hücrenin bir **adresi** ve bir **içeriği** vardır, tıpkı numaralı dolaplar gibi. **Giriş/çıkış** (G/Ç) birimleri klavye, ekran ve disktir; **veri yolu** (bus) bileşenler arasında adres, veri ve denetim sinyallerini taşır. Benzetmeyle CPU tarifi uygulayan aşçı, PC tarifte kalınan satırdır.

Şekil 1.2
Von Neumann mimarisi: CPU, ana bellek, giriş/çıkış ve sistem veri yolu



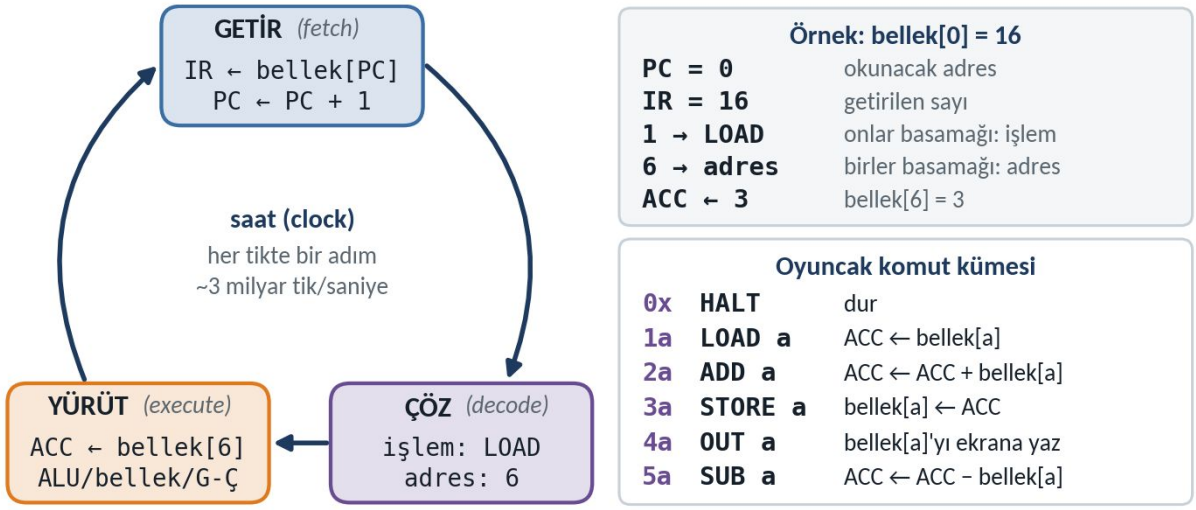
Mimarinin kilit fikri **depolanmış program** (stored program) kavramıdır: Program da veri gibi bellekte **sayılar** olarak saklanır. İşlemci bir hücredeki sayıyı, program sayacı onu gösteriyorsa komut, bir komut onu istiyorsa veri olarak yorumlar; hücre "ben komutum" diye bir etiket taşımaz. Harf, resim, ses ve program bellekte aynı biçimde, **bit örüntüleri** olarak durur; anlamı onu okuyan program verir.

> MERAK KUTUSU · **Harvard mimarisi ve iki L1 önbellek**
>
> Hopper'ın çalıştığı Harvard Mark I komutlarını delikli şeritten, verilerini ayrı bir bellekten okuyordu; bu düzene **Harvard mimarisi** denir. Günümüz işlemcileri en içte iki düzeni birleştirir: Bu makinede `lscpu`, veri ve komut için ayrı L1 önbellekler listeler: `L1d: 96 KiB (2 instances)`, `L1i: 64 KiB (2 instances)`.

3.3 Getir–çöz–yürüt döngüsü ve oyuncak bilgisayar

İşlemci açık kaldığı sürece üç adımı tekrarlar (Şekil 1.3): **Getir** (fetch): PC'nin gösterdiği adresteki komutu IR'ye getir ve PC'yi ilerlet. **Çöz** (decode): İşlemi ve adresi belirle. **Yürüt** (execute): İşlemi yap; gerekirse ALU'yu, belleği ya da G/Ç birimini kullan. Döngüyü işlemcinin **saati** (clock) yönetir; 3 GHz'lik bir işlemcide saat saniyede yaklaşık üç milyar kez "tik" atar.

Şekil 1.3
Getir–çöz–yürüt döngüsü ve oyuncak komut kümesi



Gerçek komutları izlemek başlangıçta zordur; bu yüzden **oyuncak bir bilgisayar** tanımlayalım: 8 bellek hücresi (adres 0–7) ve üç yazmaç (PC, IR, ACC). Her komut iki basamaklı bir sayıdır: **Onlar basamağı işlem kodunu, birler basamağı adresi** gösterir; 16, "1 = LOAD, 6 = adres", yani "bellek[6]'yı ACC'ye yükle" demektir. Şekil 1.4'teki program 3 ile 4'ü toplayıp sonucu yazar; Tablo 1.1 onu tur tur izler.

Şekil 1.4
8 hücrelik oyuncak bellekte iki sayıyı toplayan program



Tablo 1.1
İnsan CPU: oyuncak programın izleme tablosu

Tur	PC	IR	Komut	Yürütme	ACC	bellek[5]	Çıktı
1	0 → 1	16	LOAD 6	ACC ← bellek[6]	**3**	0	—
2	1 → 2	27	ADD 7	ACC ← ACC + bellek[7]	**7**	0	—
3	2 → 3	35	STORE 5	bellek[5] ← ACC	7	**7**	—
4	3 → 4	45	OUT 5	ekrana bellek[5]	7	7	**7**
5	4 → 5	00	HALT	dur	7	7	—

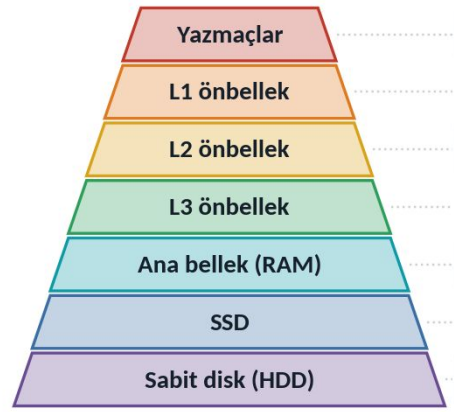
Üç gözlem: PC komut **getirilir getirilmez** artar. 4 ve 5 numaralı hücrelerin ikisi de `00` içerir; 4 komut olarak okunduğunda HALT, 5 veri olarak okunduğunda 0 anlamına gelir. "Koşula göre atla" gibi komutları H2 ve H5'te göreceğiz.

> ALGORİTMA ATÖLYESİ · **Sınıf etkinliği: İnsan CPU**
>
> Tahtaya 8 hücrelik bellek çizilir. Bir öğrenci **PC**'yi (adres kartı), biri **kontrol birimini** (sayıyı işlem ve adrese ayırır), biri **ALU/ACC**'yi, sekiz öğrenci **bellek hücrelerini**, biri de **ekranı** canlandırır ve program Tablo 1.1'deki gibi yürütülür. İkinci turda bellek[7] = 9 yapılır ve sınıf çıktıyı önce **tahmin eder** (12). Üçüncü turda 1 numaralı hücreye `57` (SUB 7) yazılır; sonucun negatif çıkması "oyuncak bellek negatif sayıyı nasıl saklar?" sorusunu H3'e bırakır. Web sayfasındaki Oyuncak CPU simülasyonu aynı programı adım adım yürütür.

3.4 Bellek hiyerarşisi

İdeal bellek hem çok hızlı, hem çok büyük, hem de ucuz olurdu; üçü birden mümkün olmadığı için bilgisayarlar bir **hiyerarşi** kurar (Şekil 1.5). Tepede CPU içindeki yazmaçlar tek bir saat çevriminde okunur; aşağı indikçe kapasite artar, hız ve bayt başına maliyet düşer. Sezgi için bir çevrimi (≈ 0,33 ns) **bir saniyeye** büyütelim: Yazmaç bir saniye, L1 önbellek birkaç saniye, RAM beş dakika, SSD üç buçuk gün, sabit disk neredeyse bir yıl sürer. Kütüphane benzetmesiyle yazmaç elinizdeki not, önbellek masanızdaki kitaplar, RAM odadaki kitaplık, disk şehrin öbür ucundaki depodur.

Şekil 1.5
Bellek hiyerarşisi: kapasite, erişim süresi ve insan ölçeğinde karşılıkları (tipik değerler)



	Kapasite	Erişim süresi	Çevrim	1 çevrim = 1 sn olsaydı
Yazmaçlar	~ yüzlerce bayt	< 1 ns	1	1 saniye
L1 önbellek	32–64 KB	~1 ns	~4	4 saniye
L2 önbellek	0,25–2 MB	~4 ns	~12	12 saniye
L3 önbellek	8–64 MB	~15 ns	~40	40 saniye
Ana bellek (RAM)	8–64 GB	~100 ns	~300	5 dakika
SSD	0,5–4 TB	~100 µs	~300 000	3,5 gün
Sabit disk (HDD)	1–20 TB	~10 ms	~30 000 000	≈ 1 yıl

Tipik mertebeler; donanıma göre değişir (~3 GHz, 1 çevrim ≈ 0,33 ns)

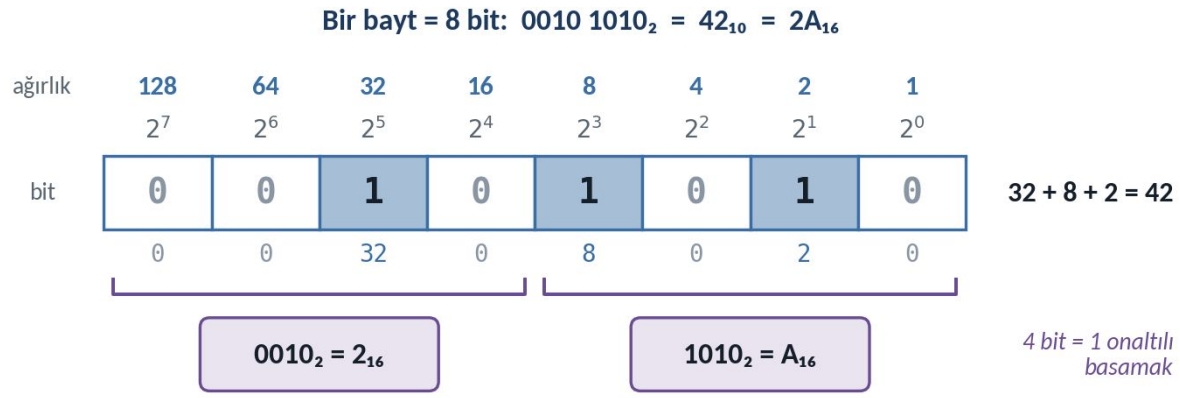
İkinci ayrım **kalıcılıktır**: Yazmaçlar, önbellekler ve RAM **uçucudur** (volatile), elektrik kesilince içerikleri kaybolur; SSD ve sabit disk **kalıcıdır**. Programlar diskte saklanır, çalıştırılacakları zaman RAM'e yüklenir (Bölüm 3.7, yükleyici).

> KAPUTUN ALTINDA · **Bu makinenin önbellekleri (lscpu)**
>
> Bu belgenin üretildiği sunucuda `lscpu`: `L1d: 96 KiB (2 instances)`, `L1i: 64 KiB (2 instances)`, `L2: 4 MiB (2 instances)`, `L3: 260 MiB (1 instance)` (KiB = 1024 bayt): "2 instances" iki çekirdektir, her birinde 48 KiB L1 veri ve 2 MiB L2 vardır, L3 ortaktır; 260 MiB'lık L3 sunucu işlemcisine özgüdür (dizüstülerde 8–32 MB). Kendi makinenizde: `lscpu` (Linux/WSL), `sysctl hw.l1dcachesize` (macOS), Görev Yöneticisi → Performans → CPU (Windows).

3.5 Verinin temsili: bit, bayt, ikili ve onaltılı

Transistör iki durumu güvenilir biçimde ayırt eder: iletir ya da iletmez. Bu yüzden en küçük bilgi birimi iki değer alan **bit**tir (binary digit). Sekiz bit bir **bayt** eder; bellek bayt bayt adreslenir. n bitle 2^n farklı örüntü yazılır: 4 bitle 16, 8 bitle 256 (0–255). Onluk tabanda `305` = 3·100 + 0·10 + 5·1'dir; **ikili** (binary) tabanda ağırlıklar 2'nin kuvvetleridir: `0010 1010₂` sayısında 1 olan bitlerin ağırlıkları 32, 8 ve 2'dir, toplam 42'dir (Şekil 1.6).

Şekil 1.6
Bir baytın konumsal değeri: $0010\ 1010_2 = 42_{10} = 2A_{16}$



İkili sayılar uzun ve okunaksızdır. **Onaltılı** (hexadecimal) tabanda $16 = 2^4$ olduğu için her 4 bit (yarım bayt) tam bir onaltılı basamağa karşılık gelir; basamaklar 0–9 ve A–F (10–15) ile yazılır. C'de onaltılı sabitler `0x` önekiyle yazılır: `0x2A`. Tablo 1.2'yi ezberlemek, dönem boyunca adresleri ve bellek dökümlerini okumayı çok kolaylaştırır.

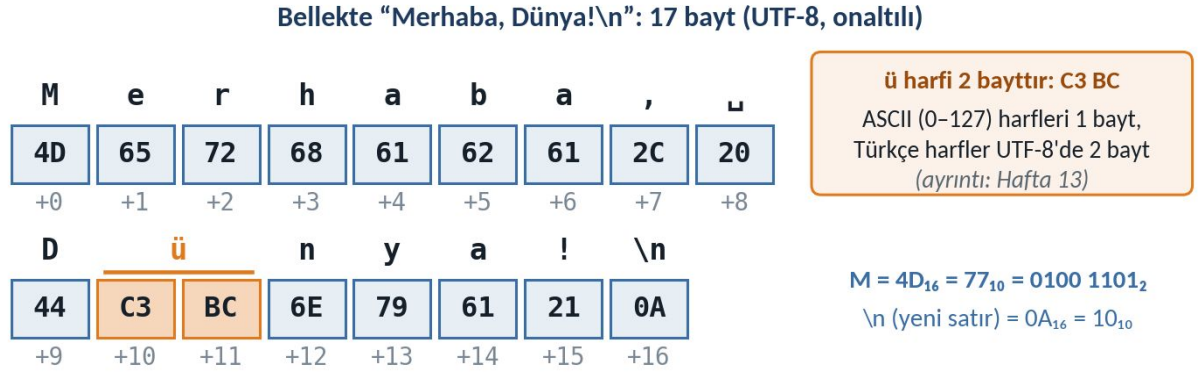
Tablo 1.2
0–15 arası değerlerin onluk, ikili ve onaltılı karşılıkları

Onluk	İkili	Onaltılı	Onluk	İkili	Onaltılı
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

1. **İkiliden onluğa:** 1 olan bitlerin ağırlıklarını toplayın: `1100 1000₂` = 128 + 64 + 8 = 200.
2. **Onluktan ikiliye:** Sürekli 2'ye bölüp kalanları yazın, kalanları **aşağıdan yukarıya** okuyun: 42 → 21 (k 0) → 10 (k 1) → 5 (k 0) → 2 (k 1) → 1 (k 0) → 0 (k 1); yukarı doğru `101010₂`.
3. **İkili ↔ onaltılı:** Bitleri sağdan dörderli gruplayıp Tablo 1.2'den çevirin: `1100 1000₂` = `C8₁₆`.
4. **Onaltılıdan onluğa:** Basamakları 16'nın kuvvetleriyle çarpıp toplayın: `0x2A` = 2·16 + 10 = 42; `0xFF` = 255.

Metin de sayıdır. ASCII tablosu 0–127 arasındaki sayıları harflere, rakamlara ve denetim karakterlerine eşler: 'A' = 65 = 0x41, 'a' = 97 = 0x61, '0' = 0x30, boşluk = 0x20, yeni satır (`\n`) = 0x0A. Büyük ve küçük harf yalnızca tek bir bitte ($2^5 = 32$) farklıdır. Türkçe karakterler ASCII'de yoktur; yaygın **UTF-8** kodlaması onları iki baytla gösterir. "Merhaba, Dünya!\n" 16 karakterdir ama 17 bayt tutar, çünkü 'ü' = `C3 BC` (Şekil 1.7).

Şekil 1.7
"Merhaba, Dünya!\n" dizisinin bellekteki UTF-8 baytları (onaltılı)

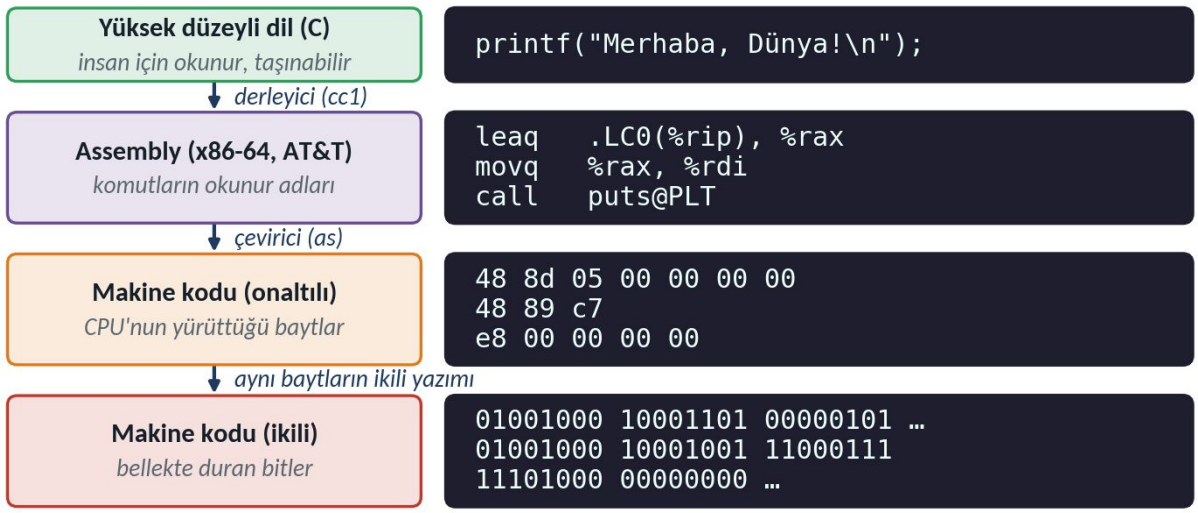


> KAPUTUN ALTINDA · **Baytları kendi gözünüzle görün: od**
>
> `od -A d -t x1` komutu kaynak dosyadaki metnin baytlarını bu makinede `4d 65 72 68 61 62 61 2c 20 44 c3 bc 6e 79 61 21` olarak döker (PowerShell'de `Format-Hex`). C17'de ikili sabit (`0b101010`) standart değildir; GCC `-pedantic` ile uyarır, C23 standartlaştırmıştır. Bu derste `0x` yazımını kullanıyoruz.

3.6 Makine dilinden C'ye

İşlemci yalnızca **makine kodunu**, yani ISA'nın tanımladığı bayt dizilerini yürütür. İlk bilgisayarlar bu sayılarla programlandı; sonra her komuta okunabilir bir ad verildi (`mov`, `add`, `call`). Bu adlarla yazılan dile **assembly**, onu makine koduna çeviren programa **çevirici** (assembler) denir; assembly işlemciye özgüdür. **Yüksek düzeyli diller** işlemciden bağımsızdır ve çeviriyi derleyiciye ya da yorumlayıcıya bırakır. Şekil 1.8, bu makinede derlediğimiz `printf` satırının dört düzeydeki gerçek karşılığıdır.

Şekil 1.8
Aynı işlem dört düzeyde: C, assembly, onaltılı ve ikili makine kodu (gerçek gcc ve objdump çıktısı)



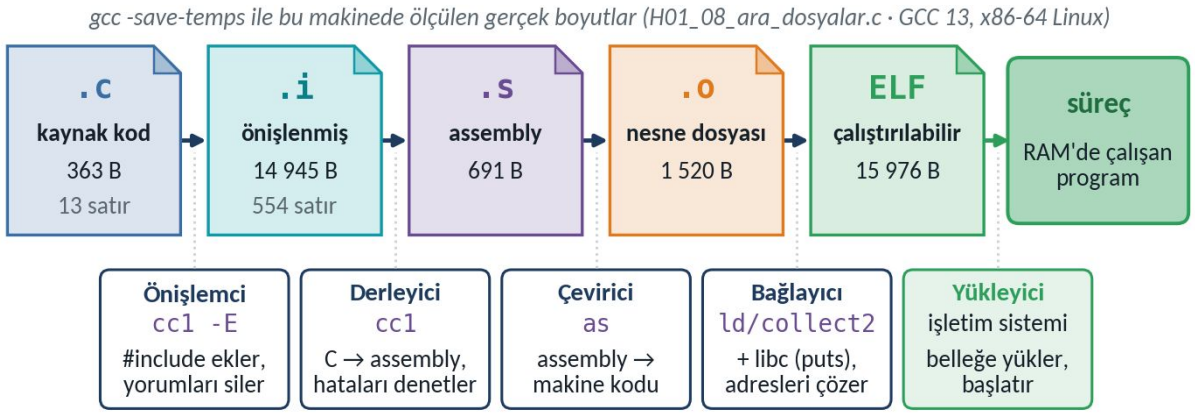
Derlenen dillerde (C, C++, Rust, Go) program çalıştırılmadan önce bir kez makine koduna çevrilir: Genellikle hızlıdır ve pek çok hata derlerken yakalanır, ama çalıştırılabilir dosya yalnızca derlendiği platformda çalışır. **Yorumlanan** dillerde (Python, JavaScript) yorumlayıcı programı çalışırken çevirir; dosya her yerde çalışır ama hatalar çoğu zaman o satır çalışınca görülür. Java ara yol izler: Kod önce **bayt koduna** (bytecode) derlenir, bu kod bir sanal makinede (JVM) çalışır.

C, 1970'lerin başında Bell Laboratuvarları'nda **Dennis Ritchie** tarafından, Ken Thompson ile geliştirdikleri UNIX'i yazmak için tasarlandı; 1973'te UNIX çekirdeğinin büyük bölümü C ile yeniden yazıldı (Ritchie, 1993). 1978'de yayımlanan ve **K&R** diye anılan kitap uzun yıllar dilin fiilî tanımı oldu (ikinci baskı: Kernighan ve Ritchie, 1988). Dil 1989'da ANSI (C89), 1990'da ISO (C90) standardı oldu; ardından C99, C11, C17 ve C23 (ISO/IEC 9899:2024) geldi. Bu derste **C17** (ISO/IEC 9899:2018) kullanıyoruz (ISO/IEC, 2018); `-std=c17` bayrağı bu yüzdendir. Linux çekirdeği, gömülü sistemler, aygıt sürücüleri, SQLite, Git ve Python yorumlayıcısı (CPython) büyük ölçüde C ile yazılmıştır; C++, Java, C# ve JavaScript'in sözdizimi de C'den türemiştir.

3.7 Derleme hattı

`gcc merhaba.c -o merhaba` tek bir işlem gibi görünür; oysa `gcc` bir **sürücü** (driver) programdır ve arka planda dört aşamayı sırayla çalıştırır, beşinci aşamayı (yükleme) işletim sistemi yapar (Şekil 1.9, Tablo 1.3). `-save-temps` bayrağı ara dosyaları saklar; şekildeki boyutlar Örnek 8'de bu makinede ölçülmüştür.

Şekil 1.9
Derleme hattı: her aşamanın girdi ve çıktı dosyaları (gerçek boyutlar)



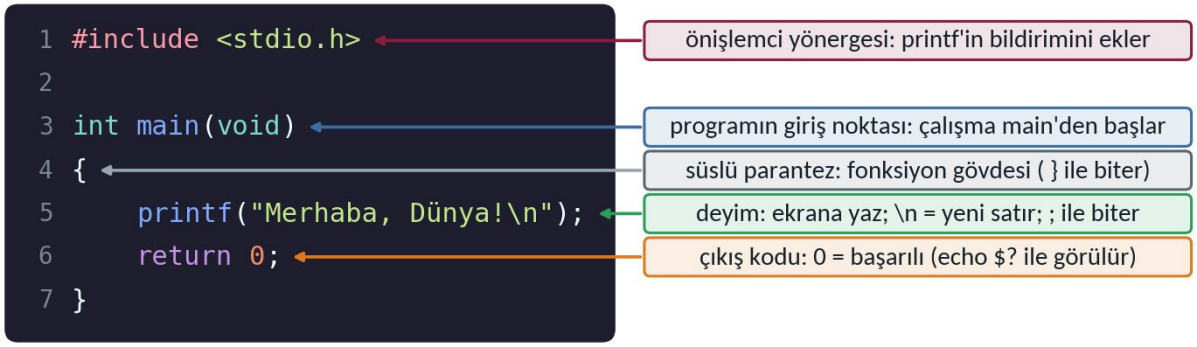
Tablo 1.3
Derleme hattının aşamaları ve GCC'deki karşılıkları

Aşama	GCC aracı	Tek başına	Girdi → çıktı	Ne yapar?
Önişlemci (preprocessor)	`cc1 -E`	`gcc -E x.c -o x.i`	.c → .i	`#include` dosyalarını ekler, `#define` adlarını yerine koyar, yorumları siler.
Derleyici (compiler)	`cc1`	`gcc -S x.i`	.i → .s	Dilin kurallarını denetler, **hata ve uyarı** üretir, C'yi assembly'ye çevirir.
Çevirici (assembler)	`as`	`gcc -c x.s`	.s → .o	Makine koduna çevirir; başka dosyadaki adlar (`puts`) için yer boş kalır.
Bağlayıcı (linker)	`ld` (`collect2`)	`gcc x.o -o x`	.o + libc → ELF/EXE	Dosyaları ve kütüphaneleri birleştirir, boş adresleri doldurur.
Yükleyici (loader)	işletim sistemi	`./x`	dosya → süreç	Belleğe yükler, libc'yi bağlar, başlangıç kodunu ve `main`'i çalıştırır.

> KAPUTUN ALTINDA · **gcc hangi programları çağırıyor?**
>
> `gcc -v` bu makinede sırasıyla `cc1` → `as` → `collect2` programlarını çalıştırır: GCC'de önişleme ve derleme aynı program (`cc1`) içindedir; `collect2` asıl bağlayıcı `ld`'yi çağırır. `ldd` ise programın yüklenirken ihtiyaç duyduğu paylaşılan kütüphaneleri gösterir:
>
> ```
> $ ldd ./ara_dosyalar   (yükleme adresleri gizlendi)
>       linux-vdso.so.1
>       libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
>       /lib64/ld-linux-x86-64.so.2
> ```
>
> `libc.so.6` C standart kütüphanesidir; `ld-linux-x86-64.so.2` onu belleğe yükleyen **dinamik yükleyicidir**. Windows/MinGW'de çalıştırılabilir dosya **PE** biçiminde ve `.exe` uzantılıdır, kütüphaneler `.dll` dosyalarıdır (MSYS2 UCRT64'te C çalışma zamanı `ucrtbase.dll`); aşamaların mantığı aynıdır.

3.8 İlk C programının anatomisi

Şekil 1.10
İlk C programının anatomisi



- `#include <stdio.h>` bir **önişlemci yönergesidir** (`#` ile başlar, `;` ile bitmez): Standart giriş/çıkış başlık dosyasını ekler; içinde `printf`'in **bildirimi** (declaration) vardır. `printf`'in makine kodu ise libc'dedir ve bağlayıcı ekler.
- `int main(void)` programın **giriş noktasıdır**. `int`, `main`'in işletim sistemine bir tamsayı döndüreceğini; `void`, parametre almadığını söyler. C büyük–küçük harfe duyarlıdır: `Main` başka bir addır.
- `{ }` fonksiyonun **gövdesidir**. K&R biçimini izliyoruz: Fonksiyonun açılış parantezi ayrı satırda, gövde dört boşluk içeride.
- `printf("…\n");` bir **deyimdir** (statement) ve `;` ile biter. Tırnak içi bir **karakter dizisi sabitidir** (string literal); `\n` tek bir karakteri (yeni satır) gösteren bir **kaçış dizisidir** (escape sequence).
- `return 0;` işletim sistemine **çıkış kodu** (exit status) olarak 0 döndürür: 0 "başarılı", 0 dışı "bir sorun oldu" demektir.

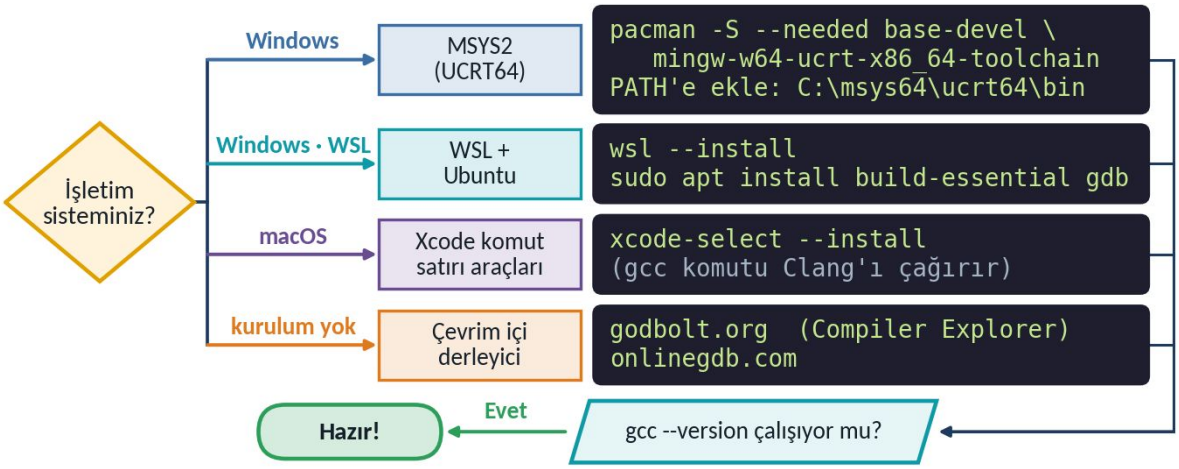
Tablo 1.4
Sık kullanılan kaçış dizileri ve printf'teki %%

Yazım	Anlamı	Örnek	Ekranda
`\n` · `\t`	yeni satır · sekme (sonraki sekme durağına atla)	`printf("A\tB\n");`	A B
`\"` · `\\`	çift tırnak · ters eğik çizgi	`printf("\"C:\\h01\"");`	"C:\h01"
`\'`	tek tırnak (karakter sabitlerinde, H3)	`'\''`	'
`%%`	yüzde (printf kuralı; kaçış dizisi değil)	`printf("%%100");`	%100

3.9 Geliştirme ortamı

Derleyici olarak **GCC**, düzenleyici olarak **C/C++** eklentisiyle **VS Code** öneriyoruz (Şekil 1.11). Windows'ta MSYS2 (msys2.org) kurulur, **UCRT64** terminalinde şekildeki `pacman` komutu çalıştırılır, `C:\msys64\ucrt64\bin` **PATH** değişkenine eklenir (Sistem → Gelişmiş sistem ayarları → Ortam Değişkenleri) ve yeni bir komut isteminde `gcc --version` ile doğrulanır. WSL, ileride kullanacağımız AddressSanitizer ve Valgrind için önerilir. İlk haftalarda derlemeyi tümleşik terminalde **elle** yapın.

Şekil 1.11
İşletim sistemine göre geliştirme ortamı kurulum akışı



Tablo 1.5
Bu hafta kullanılan GCC bayrakları

Bayrak	Anlamı
`-std=c17 -pedantic`	C17 standardına göre derle; standarda aykırı her yazımda uyar.
`-Wall -Wextra`	Olası hatalara karşı uyarıları aç. Ders örneklerinin hiçbiri uyarı üretmez.
`-o ad` · `-g`	Çıktı dosyasının adı (verilmezse `a.out` / `a.exe`) · hata ayıklayıcı bilgisi (H6).
`-E` · `-S` · `-c`	Hattı önişlemci / derleyici / çeviriciden sonra durdur (.i / .s / .o).
`-save-temps` · `-O0`/`-O2`	Ara dosyaları sakla · eniyileme (optimizasyon) düzeyi (Örnek 9).

> HATA AVCISI · **Windows konsolunda bozuk Türkçe karakterler**
>
> Örneklerimiz UTF-8 ile kaydedilir. Windows'un klasik komut isteminde kod sayfası çoğu zaman Türkçe OEM 857'dir; `Merhaba, Dünya!` baytları bu tabloyla okununca ekranda `Merhaba, D╝nya!` görünür ('ü'nün iki baytı C3 ve BC iki çizgi karakterine dönüşür). **Çözüm:** Çalıştırmadan önce `chcp 65001` (Windows Terminal'de genellikle gerekmez); dosyayı UTF-8 kaydedin (VS Code'un sağ alt köşesi). Bu bir **derleme hatası değildir**: Program doğrudur, sorun ekranın baytları yorumlamasındadır.

4. Çözümlü Örnekler: Örnek Merdiveni

Dokuz örnek bir merdiven oluşturur; her basamak **tek bir yeni öge** ekler: program iskeleti (1), kaçış dizileri (2), hizalama (3), `%d` (4), derleyici iletisi (5), derleyici–bağlayıcı ayrımı (6), çıkış kodu (7), ara dosyalar (8), eniyileme (9). ★ çekirdek, ★★ derinleştirme, ★★★ isteğe bağlıdır; Örnek 6 ve 8 amiral örnektir (şablonun tamamı). Önce **çıktıyı tahmin edin**, sonra çalıştırın; çıktılar `kodlar/` klasöründeki kodların bu makinede alınmış gerçek çıktılarıdır.

4.1 Örnek 1 ★ · Merhaba, Dünya!

Problem. Ekrana "Merhaba, Dünya!" yazan ve başarıyla bittiğini bildiren en kısa C programını yazın, derleyin, çalıştırın. İlk üç satır bir **yorumdur** (comment): `/*` ile `*/` arası derleyicice yok sayılır; ders kodlarının hepsinde dosya adı, derleme komutu ve beklenen çıktı buradadır. Sonraki örneklerde bu başlık yorumunu atlıyor, kodu dosyadaki satır numaralarıyla gösteriyoruz.

H01_01_merhaba_dunya.c

```c
/* H01_01_merhaba_dunya.c · Hafta 1 · Örnek 1: İlk C programı
 * Derle: gcc -std=c17 -Wall -Wextra -pedantic H01_01_merhaba_dunya.c -o merhaba
 * Çıktı: Merhaba, Dünya! */
#include <stdio.h>

int main(void)
{
    printf("Merhaba, Dünya!\n");
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_01_merhaba_dunya.c -o merhaba
$ ./merhaba
Merhaba, Dünya!
```

Derleme komutunun hiçbir şey yazdırmaması iyi haberdir: Ne hata ne uyarı vardır. `./` "bu klasördeki" demektir; Windows'ta `merhaba.exe` ya da `merhaba` yazmak yeterlidir. **Değiştir:** `\n`'yi silin: Kabuk istemi nereye yazılıyor? (`return 0;` silinse de C99'dan beri `main` 0 döndürür; bu derste yine de hep açıkça yazacağız.)

4.2 Örnek 2 ★ · Çok satırlı çıktı ve kaçış dizileri

Problem. Küçük bir tablo, tırnak içinde bir metin, bir Windows klasör yolu ve bir yüzde değeri yazdırın. **Yeni öge:** kaçış dizileri.

H01_02_kacis_dizileri.c · satır 4–15

```c
#include <stdio.h>

int main(void)
{
    printf("Hafta\tKonu\n");              /* \t : sekme        */
    printf("1\tBilgisayar ve program\n");
    printf("2\tAlgoritmik düşünme\n\n");  /* \n\n : boş satır  */
    printf("Tırnak: \"C dili\"\n");       /* \" : çift tırnak  */
    printf("Yol   : C:\\kodlar\\h01\n");  /* \\ : ters çizgi   */
    printf("Oran  : %%100\n");            /* %% : printf kuralı */
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_02_kacis_dizileri.c -o kacis
$ ./kacis
Hafta   Konu
1       Bilgisayar ve program
2       Algoritmik düşünme

Tırnak: "C dili"
Yol   : C:\kodlar\h01
Oran  : %100
```

`\t` sabit sayıda boşluk eklemez, imleci bir sonraki **sekme durağına** (8 sütunun katları) taşır; bu yüzden "Konu" sütunu hizalı başlar. `\n\n` bir boş satır bırakır. `%%` bir kaçış dizisi değil, `printf`'in kuralıdır. **Değiştir:** "Hafta" yerine "Hafta no" yazın: Hizalama neden bozuldu? ("Hafta no" 8 karakterdir.)

4.3 Örnek 3 ★ · ASCII sanatıyla isim kartı

Problem. Ders adını, haftayı, öğrencinin adını ve bölümünü içeren, kenarları çizgili bir isim kartı yazdırın; bütün satırlar aynı genişlikte olmalı. **Yeni öge:** çıktı düzeni ve hizalama.

H01_03_isim_karti.c · satır 4–17

```c
#include <stdio.h>

int main(void)
{
    printf("+=========================================+\n");
    printf("|   ____                                  |\n");
    printf("|  / ___|   Algoritma ve Programlama - I   |\n");
    printf("| | |       Hafta 1 · İlk C programım      |\n");
    printf("| | |___                                  |\n");
    printf("|  \\____|   Öğrenci : Ada Yılmaz           |\n");
    printf("|           Bölüm   : Bilgisayar Müh.     |\n");
    printf("+=========================================+\n");
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_03_isim_karti.c -o kart
$ ./kart
+========================================+
|   ____                                 |
|  / ___|   Algoritma ve Programlama - I  |
| | |       Hafta 1 · İlk C programım    |
| | |___                                 |
|  \____|   Öğrenci : Ada Yılmaz         |
|           Bölüm   : Bilgisayar Müh.    |
+========================================+
```

Hizalamanın sırrı, her satırın **ekranda** aynı sayıda sütun kaplamasıdır. Vurgulu satırda kaynak kodda `\\ ____|`, ekranda `\____|` görünür: Ters eğik çizgi kaynakta iki kez yazılır, bu yüzden o satır kaynakta bir karakter uzundur ama ekranda eşittir. "Öğrenci" UTF-8'de 9 bayt tutar ama 7 sütun kaplar (Ö ve ğ ikişer bayttır); hizalama bayta değil, görünen karaktere göre yapılır. **Değiştir:** Kartı kendi bilgilerinizle doldurun ve bir e-posta satırı ekleyin.

4.4 Örnek 4 ★ · printf ile ilk hesaplama

Problem. Bir ifadeyi metin olarak yazdırmak ile değerini hesaplatıp yazdırmak arasındaki farkı gösterin; bir günde kaç saniye olduğunu hesaplayın. **Yeni öge:** `%d` biçim belirteci (ayrıntısı H3'te).

H01_04_ilk_hesaplama.c · satır 4–13

```c
#include <stdio.h>

int main(void)
{
    printf("3 + 4\n");                /* metin: olduğu gibi yazılır */
    printf("%d\n", 3 + 4);            /* ifade: önce hesaplanır     */
    printf("%d + %d = %d\n", 12, 30, 12 + 30);
    printf("Bir günde %d saniye var.\n", 24 * 60 * 60);
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_04_ilk_hesaplama.c -o hesap
$ ./hesap
3 + 4
7
12 + 30 = 42
Bir günde 86400 saniye var.
```

`printf`'in ilk argümanı bir **biçim dizisidir** (format string): Sıradan karakterler olduğu gibi yazılır; `%d` ise "buraya, virgülden sonraki tamsayının onluk değerini koy" demektir. İlk satırda `3 + 4` tırnak içindedir, yani metindir; ikincide tırnak dışındadır: Önce hesaplanır, sonuç (7) `%d`'nin yerine yazılır. Üçüncü satırdaki üç `%d` sırasıyla 12, 30 ve 42 ile eşleşir. **Tahmin et:** `printf("%d\n", 2 * 60 * 60);` ile `printf("2 * 60 * 60\n");` ne yazdırır?

4.5 Örnek 5 ★ · Noktalı virgülü eksik program

Problem. Aşağıdaki program derlenmiyor; iletiyi okuyarak hatanın yerini ve nedenini bulun. **Yeni öge:** derleyici hata iletisinin yapısı. (Kasıtlı olarak hatalı Hata Avcısı dosyaları `kodlar/hatali/` altındadır; iletilerdeki satır numaraları değişmesin diye "BU DOSYA KASITLI HATA İÇERİR" işareti dosyanın sonundaki yorumdadır.)

hatali/H01_05_noktali_virgul.c · satır 1–7

```c
#include <stdio.h>

int main(void)
{
    printf("Merhaba, Dünya!\n")
    return 0;
}
```

TERMİNAL (GERÇEK GCC 13 İLETİSİ)

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_05_noktali_virgul.c -o program
H01_05_noktali_virgul.c: In function 'main':
H01_05_noktali_virgul.c:5:32: error: expected ';' before 'return'
    5 |     printf("Merhaba, Dünya!\n")
      |                                ^
      |                                ;
    6 |     return 0;
      |     ~~~~~~
$ echo $?
1
```

GCC iletileri hep aynı kalıptadır: `dosya:satır:sütun: tür: açıklama`. "5. satırın 32. sütununda, `return`'den önce `;` bekleniyordu." Derleyici düzeltmeyi bile önermiştir (alttaki `;`). **error** türü derlemenin durduğunu, programın **üretilmediğini** gösterir (`echo $?` → gcc'nin çıkış kodu 1). Derleyici eksikliği bir sonraki sözcüğü (`return`) görünce fark eder; bu yüzden iletideki satırda hata göremiyorsanız bir önceki satıra bakın. Kapanış `}` unutulursa (`hatali/H01_05b_suslu_parantez.c`) eksiklik ancak dosya sonunda anlaşılır: `H01_05b_suslu_parantez.c:6:5: error: expected declaration or statement at end of input`.

4.6 Örnek 6 ★★ · Derleyici mi, bağlayıcı mı? (amiral)

1. Problem. İki arkadaşınız "Merhaba" programını derleyemiyor: Biri `printf` yerine `prinft`, öteki `main` yerine `Main` yazmış. Her programın **hangi aşamada** ve **neden** başarısız olduğunu belirleyin; hataları sınıflandırmak için genel bir yöntem geliştirin. **2. Çözümleme** Tablo 1.6'dadır.

Tablo 1.6
Örnek 6'nın girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• Hatalı `.c` dosyası • GCC 13 (`-std=c17 -Wall -Wextra -pedantic`)	• Derleme ile bağlamayı ayrı çalıştır (`-c`, sonra bağla) • Her aşamanın iletisini ve çıkış kodunu incele; `nm` ile adlara bak	• Hatanın aşaması: derleyici / bağlayıcı / çalışma • Neden ve düzeltme

3. Algoritma. (1) `gcc -c dosya.c` ile **yalnızca derle**; ileti `error` içeriyorsa hata **derleyicidedir**. (2) Derleme başarılıysa (uyarı olsa bile) `gcc dosya.o -o program` ile **bağla**; `undefined reference` ile başarısız olursa hata **bağlayıcıdadır**. (3) O da başarılıysa **çalıştır**; yanlış davranış çalışma zamanına ya da mantığa aittir. **4. Kod.** `H01_06b_main_buyuk_harf.c`, Örnek 1'in `main` yerine `Main` yazılmış hâlidir; `prinft` dosyası:

hatali/H01_06a_prinft.c · satır 1–7

```c
#include <stdio.h>

int main(void)
{
    prinft("Merhaba, Dünya!\n");
    return 0;
}
```

5. Tek adımda derleme. Öğrencinin gördüğü ileti şudur:

TERMİNAL (GERÇEK GCC 13 İLETİSİ)

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_06a_prinft.c -o program
H01_06a_prinft.c: In function 'main':
H01_06a_prinft.c:5:5: warning: implicit declaration of function 'prinft'; did you
mean 'printf'? [-Wimplicit-function-declaration]
    5 |     prinft("Merhaba, Dünya!\n");
      |     ^~~~~~
      |     printf
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
H01_06a_prinft.c:(.text+0x18): undefined reference to `prinft'
collect2: error: ld returned 1 exit status
$ echo $?
1
```

İletide iki farklı ses vardır. İlk satırlar **derleyiciden** gelen bir **uyarıdır** (warning): `prinft`'in bildirimi görülmemiştir; derleyici "belki `printf`?" der ama devam eder. `/usr/bin/ld:` ile başlayan satırlar **bağlayıcıdandır**: Derleyicinin boş bıraktığı `prinft` çağrısını bağlayıcı libc dahil hiçbir yerde bulamaz ve **tanımsız başvuru** (undefined reference) hatası verir; `collect2: error: ld returned 1 exit status` bunun imzasıdır.

6. İzleme tablosu ve 7. örnek çalıştırma. Algoritmayı adım adım uygulayalım (Tablo 1.7). `nm` bir nesne dosyasındaki adları listeler: `T` bu dosyada **tanımlı**, `U` başka yerde aranacak **tanımsız** addır.

TERMİNAL (GERÇEK ÇIKTI)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -c H01_06a_prinft.c
(yukarıdaki uyarı; H01_06a_prinft.o üretildi, çıkış kodu 0)
$ nm H01_06a_prinft.o
0000000000000000 T main
                 U prinft
$ gcc H01_06a_prinft.o -o program
/usr/bin/ld: H01_06a_prinft.o: in function `main':
H01_06a_prinft.c:(.text+0x18): undefined reference to `prinft'
collect2: error: ld returned 1 exit status
```

Tablo 1.7
Örnek 6: hataların aşamalara göre izlenmesi

Dosya	1) gcc -c	nm	2) bağlama	Sonuç
H01_06a_prinft.c	uyarı; .o üretildi	T main · U prinft	**undefined reference to 'prinft'**	bağlayıcı hatası
H01_06b_main_...c	sorunsuz; .o üretildi	T Main · U printf	**undefined reference to 'main'**	bağlayıcı hatası
H01_01_merhaba_dunya.c	sorunsuz	T main · U puts	başarılı	çalışır

`Main` örneğinde derleyici hiçbir şeyden şikâyet etmez, çünkü `Main` geçerli bir addır. Ama bağlayıcı her C programının başına eklediği başlangıç kodunu (`Scrt1.o` içindeki `_start`) bağlarken `main`'i arar ve bulamaz: "(.text+0x1b): undefined reference to 'main'" (ardından yine `collect2: error: ld returned 1 exit status`).

8. Test tablosu. Yöntemi haftanın bütün hatalı dosyalarına uygulayalım (Tablo 1.8; "gerçek" sütunu bu makinede alınmıştır).

Tablo 1.8
Örnek 6: test tablosu

Dosya	Hata	Beklenen	Gerçek (GCC 13)
H01_05_noktali_virgul.c	`;` eksik	derleyici	error; program üretilmedi
H01_05b_suslu_parantez.c	`}` eksik	derleyici	error: … at end of input
H01_06a_prinft.c	yazım	uyarı + bağlayıcı	warning + undefined reference
H01_06b_main_buyuk_harf.c	`Main`	bağlayıcı	undefined reference to 'main'
H01_06c_include_eksik.c	`#include` yok	uyarı	2 uyarı; program **üretildi**, çalıştı

Son satır özellikle öğreticidir: `#include <stdio.h>` unutulunca GCC 13 yalnızca uyarır, program oluşur ve doğru çalışıyormuş gibi görünür. Derleyici `printf`'in nasıl çağrılacağını **tahmin etmiştir**; bu tahmin başka fonksiyonlarda yanlış olabilir. Uyarıları görmezden gelmemenin nedeni budur.

> NOT · **Derleyici sürümü fark eder**
>
> - **GCC 14 ve sonrası** bildirimi görülmemiş bir fonksiyonun çağrılmasını (implicit function declaration) varsayılan olarak **hata** sayar; güncel MSYS2'de `prinft` örneği derleme aşamasında durur, bağlayıcıya hiç ulaşmaz.
> - **MinGW-w64 (Windows)** üzerinde `Main` iletisi genellikle "undefined reference to 'WinMain'" biçimindedir; macOS'taki Clang ise `Undefined symbols for architecture …` yazar. İleti farklıdır, aşama aynıdır: bağlayıcı.

9. Değiştir–genişlet. (a) `prinft`'i düzeltip `nm` çıktısını yeniden alın: `printf` yerine neden `puts` görünüyor? (Yanıt Örnek 8'de.) (b) Hem `;` eksik hem `Main` hatalı bir programda kaç ileti görürsünüz? Önce tahmin edin: Derleyici hatası varken bağlayıcı hiç çalışmaz.

4.7 Örnek 7 ★★ · Çıkış kodu

Problem. `main`'in döndürdüğü değerin nereye gittiğini gösterin. **Yeni öge:** çıkış kodunun kabukta gözlenmesi.

H01_07_cikis_kodu.c · satır 4–10

```
4  #include <stdio.h>
5
6  int main(void)
7  {
8      printf("Program çalıştı; işletim sistemine 3 döndürüyor.\n");
9      return 3;   /* 0: başarılı, 0 dışı: bir sorun oldu */
10 }
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_07_cikis_kodu.c -o cikis
$ ./cikis
Program çalıştı; işletim sistemine 3 döndürüyor.
$ echo $?
3
$ ./cikis > /dev/null && echo "başarılı" || echo "başarısız"
başarısız
```

`echo $?` son programın çıkış kodunu sorar: 3. `&&`'den sonraki komut yalnızca önceki program **0** döndürürse, `||`'dan sonraki 0 dışı bir değerde çalışır; derleme betikleri ve testler başarıyı bu değerden anlar. Windows komut isteminde `echo %errorlevel%`, PowerShell'de `$LASTEXITCODE` kullanılır.

> MERAK KUTUSU · **return 256 ne döndürür?**
>
> Linux'ta kabuğun gördüğü çıkış kodu 8 bitliktir (0–255). Bu makinede `return 256;` ile biten program için `echo $?` 0, `return -1;` için 255 yazdırdı: Yalnızca en düşük 8 bit korunur (256 = 1 0000 0000_2). Windows'ta çıkış kodu 32 bitliktir. Taşınabilir programlar 0–255 aralığında kalır; standart ayrıca `EXIT_SUCCESS` ve `EXIT_FAILURE` adlarını tanımlar (H8).

4.8 Örnek 8 ★★ · gcc -save-temps ile ara dosyalar (amiral)

1. Problem. "Merhaba" programının derleme hattındaki yolculuğunu belgeleyin: Her aşamanın dosyasını saklayın, boyutunu ölçün, içeriğinden Büyük Soru'ya kanıt olacak bir kesit seçin. **2. Çözümleme** Tablo 1.9'dadır.

Tablo 1.9
Örnek 8'in girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `H01_08_ara_dosyalar.c` (363 bayt) • GCC 13, x86-64 Linux	• Önişle, derle, çevir: `-save-temps -c`; sonra bağla • Boyut (`ls -l`), satır (`wc -l`), tür (`file`) ölç • Kesit al (`tail`, `objdump`, `nm`)	• .i, .s, .o ve çalıştırılabilir dosya • Aşama başına boyut ve gözlem tablosu

3. Algoritma. (1) `-save-temps -c` ile derle (önişlemci, derleyici ve çevirici çıktıları kalır). (2) Nesne dosyasını ayrıca bağla. (3) Boyut, satır ve tür ölç. (4) Her dosyadan bir kesit alıp bir önceki aşamayla karşılaştır. İki adımda derliyoruz, çünkü GCC 11 ve sonrası tek adımlı `-save-temps -o ad` kullanımında ara dosyalara önek ekler (bu makinede `ara_dosyalar-H01_08_ara_dosyalar.i`). **4. Kod.** Önişlemcinin işini görmek için bir `#define` ekledik (sabitler H3'te):

H01_08_ara_dosyalar.c · satır 5–13

```c
#include <stdio.h>

#define MESAJ "Merhaba, Dünya!\n"  /* önişlemci MESAJ'ı metinle değiştirir */

int main(void)
{
    printf(MESAJ);
    return 0;
}
```

5. Açıklama. `#define MESAJ "…"` önişlemciye "`MESAJ`'ı gördüğün her yere bu metni koy" der; derleyici `MESAJ` adını hiç görmeyecektir. **6. Çalıştırma:**

TERMİNAL (GERÇEK ÇIKTI; BOYUT VE SATIR ÖLÇÜMLERİ İZLEME TABLOSUNDA)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -save-temps -c H01_08_ara_dosyalar.c
$ gcc H01_08_ara_dosyalar.o -o ara_dosyalar
$ tail -6 H01_08_ara_dosyalar.i
# 9 "H01_08_ara_dosyalar.c"
int main(void)
{
    printf("Merhaba, Dünya!\n");
    return 0;
}
```

TERMİNAL (GERÇEK ÇIKTI, DEVAM)

```
$ file H01_08_ara_dosyalar.o ara_dosyalar | cut -c1-95
H01_08_ara_dosyalar.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not
stripped
ara_dosyalar:          ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically lin
$ ./ara_dosyalar
Merhaba, Dünya!
```

7. İzleme tablosu (Tablo 1.10):

Tablo 1.10
Örnek 8: derleme hattının dosya dosya izlenmesi (gerçek ölçümler)

Aşama	Dosya	Boyut (B)	Gözlem
kaynak	`.c`	363	13 satır; yorum ve #define içerir
önişlemci	`.i`	**14 945**	554 satır; stdio.h eklendi, yorumlar silindi, MESAJ yerine metin kondu
derleyici	`.s`	691	x86-64 assembly; printf yerine puts çağrısı
çevirici	`.o`	1 520	makine kodu; puts'un adresi henüz 00 00 00 00
bağlayıcı	`ara_dosyalar`	**15 976**	ELF çalıştırılabilir; puts adresi doldurulduu

En şaşırtıcı satır ikincisidir: 13 satırlık kaynak, önişlemciden **554 satır** olarak çıkar, çünkü önişlemci `stdio.h`'nin ve onun içerdiği başlık dosyalarının bütün metnini eklemiştir (`printf`'in bildirimi .i dosyasının 366. satırındadır). Yukarıdaki `tail -6` çıktısında programımız görülür: Yorumlar silinmiş, `MESAJ` metinle değişmiştir; `# 9 "…"` satırları derleyiciye asıl dosyadaki satır numaralarını bildirir. Derleyicinin ürettiği assembly'nin özü (`.cfi_` yönergeleri çıkarılmıştır):

H01_08_ARA_DOSYALAR.S (KESİT)

```
.LC0:
        .string "Merhaba, D\303\274nya!"
main:
        endbr64
        pushq   %rbp
        movq    %rsp, %rbp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        ret
```

Metin salt okunur veri bölümüne konmuştur ve 'ü' `\303\274` biçimindedir: C3 BC baytlarının sekizli yazımı. Kodda `printf` değil `puts` çağrılır: Biçim belirteci içermeyen ve `\n` ile biten bir `printf`'i GCC, eniyileme kapalıyken bile daha basit olan `puts`'a çevirir. Çevirici ise bu kodu makine koduna çevirir ve bilmediği adresleri bağlayıcıya bırakır:

TERMİNAL (GERÇEK ÇIKTI)

```
$ objdump -d -r H01_08_ara_dosyalar.o   (kesit)
0000000000000000 <main>:
   8:  48 8d 05 00 00 00 00   lea    0x0(%rip),%rax        # f <main+0xf>
      b: R_X86_64_PC32  .rodata-0x4
  12:  e8 00 00 00 00         call   17 <main+0x17>
      13: R_X86_64_PLT32  puts-0x4
```

`call` komutunun (`e8`) ardındaki dört bayt `00 00 00 00`'dır ve `R_X86_64_PLT32 puts` kaydı bağlayıcıya "burayı doldur" der. Bağlanmış dosyada aynı komut gerçek bir adrese gider: `115b: call 1050 <puts@plt>`. Yolculuğun kütüphanedeki son adımı ise çekirdeğe yapılan **sistem çağrısıdır**; `strace` bunu gösterir:

TERMİNAL (GERÇEK ÇIKTI)

```
$ strace -e trace=write ./ara_dosyalar
write(1, "Merhaba, D\303\274nya!\n", 17) = 17
+++ exited with 0 +++
```

8. Test tablosu (Tablo 1.11): Raporun iddialarını denetleyelim.

Tablo 1.11
Örnek 8: rapor iddialarının denetimi

İddia	Denetim	Beklenen	Gerçek
Yorumlar silinir, MESAJ yerine metin konur	`tail -6 *.i`	printf("Merhaba...")	✓
printf, puts'a çevrilir	`grep call *.s`	call puts@PLT	✓
Nesne dosyasında puts tanımsızdır	`nm *.o`	U puts	✓
Program 17 baytlık tek bir write yapar	`strace -e trace=write`	write(1, ..., 17)	✓

9. Değiştir–genişlet. (a) `#include`'u silip .i boyutunu yeniden ölçün. (b) `printf("%s", MESAJ)` yazın: .s'de hâlâ `puts` var mı? (c) Ölçümleri kendi bilgisayarınızda tekrarlayın; Windows'ta dosya boyutu ve `file` çıktısı nasıl değişiyor?

4.9 Örnek 9 ★★★ · -O0 ile -O2 (Merak)

Problem. Aynı C programı farklı eniyileme düzeylerinde farklı makine kodlarına dönüşebilir mi? (Değişkenler H3'te; `int a = 3;` satırını "a adlı kutuya 3 koy" diye okuyun.)

H01_09_optimizasyon.c · satır 5–14

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
    int b = 4;
    int toplam = a + b;
    printf("Toplam: %d\n", toplam);
    return 0;
}
```

Tablo 1.12
main fonksiyonu: gcc -S -O0 ve -O2 çıktıları (gerçek; .cfi_ yönergeleri -fno-asynchronous-unwind-tables ile gizlendi)

	-O0 (19 komut) · 1–10	-O0 · 11–19 (devam)	-O2 (10 komut)

main:	`endbr64`
	`pushq %rbp`
	`movq %rsp, %rbp`
	`subq $16, %rsp`
	`movl $3, -12(%rbp)`
	`movl $4, -8(%rbp)`
	`movl -12(%rbp), %edx`
	`movl -8(%rbp), %eax`
	`addl %edx, %eax`
	`movl %eax, -4(%rbp)`
		`movl -4(%rbp), %eax`
		`movl %eax, %esi`
		`leaq .LC0(%rip), %rax`
		`movq %rax, %rdi`
		`movl $0, %eax`
		`call printf@PLT`
		`movl $0, %eax`
		`leave`
		`ret`
			`endbr64`
			`subq $8, %rsp`
			`movl $7, %edx`
			`movl $2, %edi`
			`xorl %eax, %eax`
			`leaq .LC0(%rip), %rsi`
			`call __printf_chk@PLT`
			`xorl %eax, %eax`
			`addq $8, %rsp`
			`ret`

`-O0` C kodunu neredeyse satır satır izler: 3 ve 4 bellekteki iki yere (`-12(%rbp)`, `-8(%rbp)`) yazılır, yazmaçlara okunur, `addl` ile toplanır, sonuç yine belleğe yazılıp okunur. `-O2`'de toplama **yoktur**: Derleyici 3 + 4'ü derleme sırasında hesaplayıp doğrudan `movl $7, %edx` yazmıştır (sabit katlama, constant folding). `endbr64`, Ubuntu GCC'sinin varsayılan eklediği bir güvenlik komutudur (Intel CET); `-O2`'deki `__printf_chk` da Ubuntu'nun eniyileme açıkken `_FORTIFY_SOURCE` denetimlerini etkinleştirmesindendir. Kodu Compiler Explorer'da (godbolt.org) farklı derleyicilerle deneyin (orada Intel sözdizimi: `mov edx, 7`). Öğrenirken `-O0 -g`, teslimde `-O2` kullanılır: Eniyilenmiş kod hızlıdır ama C satırlarıyla bire bir eşleşmez.

5. Hata Avcısı

Haftanın hataları üç gruba ayrılır: **derleyici hataları** (kod dilin kurallarına uymuyor), **bağlayıcı hataları** (kurallara uyuyor ama bir ad bulunamıyor) ve **ortam sorunları** (program doğru, çevresi yanlış ayarlı). Tablo 1.13 her hatanın yakalandığı aşamayı özetler.

Tablo 1.13
Hata Avcısı: hata, ileti, neden ve çözüm

Hata	İleti (GCC 13)	Neden	Çözüm
Eksik `;` / `}`	`expected ';' before 'return'` · `expected declaration or statement at end of input`	Deyim ya da blok kapanmamış; derleyici bunu sonraki sözcükte ya da dosya sonunda fark eder.	İletideki satırın **bir öncesine** de bakın; parantezleri eşleştirin.
`#include` yok	`warning: implicit declaration of function 'printf'`	`printf`'in bildirimi görülmedi; çağrı tahmin edildi.	`#include <stdio.h>` ekleyin; uyarıyı yok saymayın (GCC 14'te hata).
`prinft`	uyarı + “undefined reference to ‘prinft’”	Böyle bir fonksiyon yok; bağlayıcı bulamadı.	“did you mean” önerisine bakıp yazımı düzeltin.
`Main`	“undefined reference to ‘main’” (MinGW: ‘WinMain’)	C harf duyarlı; başlangıç kodu `main`'i arıyor.	`int main(void)` yazın.
Bozuk Türkçe	(ileti yok) ekranda `D├╝nya`	UTF-8 baytları 857 kod sayfasıyla okunuyor.	`chcp 65001`; dosyayı UTF-8 kaydedin.

İleti okuma alışkanlıkları. Yukarıdan başlayın: İlk hata sonrakileri doğurabilir (`printf("Merhaba\n";` iki ileti üretir, ilkini düzeltince ikincisi de kaybolur). Önce `dosya:satır:sütun`, sonra tür alanına

bakın: `error` derlemeyi durdurur, `warning` durdurmaz ama ciddiye alınır, `note` açıklamadır. `/usr/bin/ld:` ya da `collect2` görüyorsanız sorun dilbilgisinde değil adlardadır: Bağlayıcı bir şeyi bulamıyordur.

6. Mühendis Gözüyle

- **Soyutlamayı kullanın ama bir altını bilin.** `printf` yazarken `write`'ı düşünmezsiniz; ama çıktı görünmediğinde (ör. `\n` yokken tampon boşaltılmadığında) alt katmanı bilmek hatayı hızla buldurur.
- **Kaynak taşınır, ikili dosya taşınmaz.** Aynı `.c` dosyası Linux'ta ELF, Windows'ta PE olur; ilk argüman Linux x86-64'te `%rdi`, Windows x64'te `%rcx` yazmacıyla aktarılır; ARM'nin komut kümesi bambaşkadır. Standart C'ye (`-std=c17 -pedantic`) bağlı kalın.
- **Uyarılar ücretsiz hata ayıklamadır.** `-Wall -Wextra` derleyicinin yaptığı bir kod incelemesidir; profesyonel projelerde `-Werror` uyarıları hataya çevirir. Hedefimiz **sıfır uyarı**.
- **Tekrarlanabilirlik ve güvenlik.** Aynı hatalı program GCC 13'te uyarı, GCC 14'te hata verir: Raporlarda sürümü, sistemi ve bayrakları yazın. Örnek 9'daki `endbr64` ve `__printf_chk` derleyicinin sessizce eklediği güvenlik önlemleridir; güvenli kodlama alışkanlıklarını dönem boyunca (H10, H12, H13) ekleyeceğiz.

7. Mini-OBS Projesi: v0.1

Dönem boyunca her hafta yalnızca o haftanın kavramlarıyla büyüyen tek bir uygulama geliştireceğiz: **Mini-OBS (Öğrenci Not Yönetim Sistemi)**; v0.3'ten itibaren program başlığı "MİNİ-OBS v0.N · Öğrenci Not Sistemi" biçimindedir. Dönem sonunda (v1.0) öğrenci kayıtlarını yapılarda tutan, notları hesaplayıp istatistik çıkaran, verisini dosyada saklayan, menü güdümlü ve çok dosyalı bir program olacak. Duraklar: v0.3 (H3) tek öğrenci, v0.7 (H7) menü döngüsü, v0.10 (H10) diziler, v1.0 (H14) yapılar ve dosyalar. Bugün **karşılama ekranını ve ana menünün görünümünü** tasarlıyoruz. Aşağıdaki beş seçenekli menü dönem sonu ürününün (v1.0) **vizyonudur**: Menü seçimi ilk kez v0.7'de (H7) okunur; menü orada `1) Not girişi`, `2) Sınıf istatistikleri`, `3) Çıkış` seçenekleriyle çalışmaya başlar ve sürümler ilerledikçe büyüyerek v1.0'da bu vizyona ulaşır.

Gereksinimler. (1) Kenarları çizili bir karşılama kutusunda programın adı, sürümü (v0.1) ve bölümün adı yer alır. (2) Altında "ANA MENÜ" başlığı ve en az beş seçenek listelenir (öğrenci ekle, listele, not gir, istatistikler, çıkış). (3) Menü "Seçiminiz: _" satırıyla biter (seçim henüz okunmaz; v0.7). (4) Yalnızca `printf` ve kaçış dizileri kullanılır; program `return 0;` ile biter. **İpuçları:** Kutuyu önce kâğıda çizin; ASCII sanatındaki her `\` karakterini kaynakta `\\` yazın (Örnek 3); Türkçe karakterleri bayt değil sütun olarak sayın.

Hedef ekran. Programınız çalıştığında aşağıdakine benzer bir çıktı üretmelidir (kutunun içeriğini ve ASCII sanatını kendiniz tasarlayabilirsiniz). Referans çözüm öğretim üyesindedir (`91_Proje_MiniOBS/v0.1/`).

BEKLENEN EKRAN (REFERANS ÇÖZÜMÜN GERÇEK ÇIKTISI)

```
+================================================+
|   __  __ _       _         ___  ____  ____     |
|  |  \/  (_)_ __ (_)      / _ \|  _ )/ ___|    |
|  | |\/| | | '_ \| |_____| | | | |  _ \\___ \    |
|  | |  | | | | | | |_____| |_| | |_) |___) |   |
|  |_|  |_|_|_| |_|_|      \___/|____/|____/    |
|                                                |
|     MİNİ-OBS v0.1 · Öğrenci Not Sistemi        |
|     RTEÜ Bilgisayar Mühendisliği · AP-1        |
+================================================+
  ANA MENÜ
  --------------------------------
  [1]  Öğrenci ekle
  [2]  Öğrencileri listele
  [3]  Not gir
  [4]  İstatistikler
  [0]  Çıkış
  --------------------------------
  Seçiminiz: _
```

- ☐ **T1 Derleme:** `gcc -std=c17 -Wall -Wextra -pedantic mini_obs_v0_1.c -o mini_obs` hiçbir ileti üretmez.
- ☐ **T2 Görünüm:** `./mini_obs` kutuyu ve en az beş seçenekli menüyü, sağ kenarı hizalı olarak ve "Seçiminiz" satırıyla gösterir.
- ☐ **T3 Çıkış kodu:** `./mini_obs > /dev/null; echo $?` → 0.
- ☐ **T4 Windows:** `chcp 65001` sonrası `mini_obs.exe` Türkçe karakterleri doğru gösterir.

8. Alıştırmalar

A) çıktı tahmini, B) hata bulma, C) Parsons (karışık satırları sıralama), D) kod tamamlama ve sayı sistemleri, E) kod yazma. Önce kodu **çalıştırmadan** çözün, sonra bilgisayarda doğrulayın. Çözümler öğrenci sürümünde yer almaz.

A) Çıktı tahmini

Alıştırma 1.1 ★☆☆ *Çıktı tahmini*

`main` içinde yalnızca aşağıdaki satırlar bulunan bir programın ekran çıktısını, boşluk ve satır sonlarıyla birlikte tam olarak yazın.

alistirmalar/H01_A01_cikti_tahmini.c · satır 12–15

```
12      printf("Bir\tİki\n");
13      printf("Üç\\n\n");
14      printf("%d\n", 5 + 5);
15      printf("5 + 5\n");
```

Alıştırma 1.2 ★★☆ *Çıktı tahmini (İnsan CPU)*

Oyuncak bilgisayarın 8 hücresinde sırasıyla `15, 56, 37, 47, 00, 09, 04, 00` bulunuyor (5 = SUB). PC = 0'dan başlayarak izleme tablosunu doldurun. Ekrana ne yazılır? bellek[7]'nin son değeri nedir? HALT dahil kaç komut yürütülür?

B) Hata bulma

Alıştırma 1.3 ★☆☆ *Hata bul*

`#include <stdio.h>` ile başlayan programın `main`'i aşağıdadır. Program neden derlenmez? Kaç ileti göreceğinizi ve ilk iletinin hangi satırı ve sütunu göstereceğini tahmin edin.

alistirmalar/H01_A03_eksik_parantez.c · satır 13–17

```
13  int main(void)
14  {
15      printf("Merhaba\n";
16      return 0;
17  }
```

Alıştırma 1.4 ★★☆ *Hata bul*

`#include <stdio.h>` bulunan aşağıdaki program GCC 13 ile hangi aşamada, hangi iletiyle başarısız olur? GCC 14'te ne değişir?

alistirmalar/H01_A04_buyuk_harf_printf.c · satır 12–16

```
12  int main(void)
13  {
14      Printf("Sonuç: %d\n", 6 * 7);
15      return 0;
16  }
```

Alıştırma 1.5 ★★☆ *Hata bul (ileti okuma)*

İleti: `H01_A05_ileti_okuma.c:16:30: error: expected ';' before 'printf'`. İleti 16. satırı gösterdiği hâlde neden sonraki satırdaki `printf`'ten söz ediyor? Nasıl düzeltilir?

alistirmalar/H01_A05_ileti_okuma.c · satır 14–19

```
14  int main(void)
15  {
16      printf("Birinci satır\n")
17      printf("İkinci satır\n");
18      return 0;
19  }
```

C) Parsons problemleri

Alıştırma 1.6 ★☆☆ *Parsons*

Satırları ekrana `Merhaba!` yazan ve 0 ile biten geçerli bir program olacak biçimde sıralayın (girintileri de belirleyin): `return 0;` · `#include <stdio.h>` · `}` · `int main(void)` · `printf("Merhaba!\n");` · `{`

Alıştırma 1.7 ★★☆ *Parsons*

`kart.c` için derleme hattını elle yürüten komutları sıraya dizin ve her birinin ürettiği dosyayı yazın: `gcc -c kart.s -o kart.o` · `gcc -E kart.c -o kart.i` · `./kart` · `gcc kart.o -o kart` · `gcc -S kart.i -o kart.s`

D) Kod tamamlama ve sayı sistemleri

Alıştırma 1.8 ★☆☆ *Kod tamamla*

Boşlukları (__) doldurun: Ekranda `Ad:` ile `"Ada"` arasında bir sekme, ikinci satırda `C:\temp` görünmeli.

alistirmalar/H01_A08_kacis_tamamla.c · satır 13–14

```
13      printf("Ad:__\"Ada__\n");
14      printf("Klasör: C:__temp\n");
```

Alıştırma 1.9 ★☆☆ *Sayı sistemleri*

Dönüştürün ve yöntemi belirtin: (a) 25_{10} → ikili ve onaltılı, (b) $1100\ 1000_2$ → onluk ve onaltılı, (c) 0xFF → onluk, (d) 0x7F → ikili, (e) 100_{10} → onaltılı.

Alıştırma 1.10 ★★☆ *Sayı sistemleri*

(a) Bir bellek dökümündeki `4D 65 72 68 61 62 61` baytları ASCII'de hangi sözcüktür? (b) 16 bitle kaç farklı değer yazılır? (c) Bir onaltılı basamak neden tam 4 bite karşılık gelir? (d) 'A' (0x41) ile 'a' (0x61) hangi bitte farklıdır?

E) Kod yazma

Alıştırma 1.11 ★☆☆ *Kod yaz*

Örnek 3'ü temel alarak kendi isim kartınızı yazın: en az altı satır, ekranda eşit genişlik, en az bir `\` karakteri. Sıfır uyarıyla derlendiğini gösterin.

Alıştırma 1.12 ★★☆ *Kod yaz*

Bir ileti yazıp 42 çıkış koduyla biten bir program yazın ve `./program; echo $?` ile doğrulayın. `return 300;` yazarsanız kabuğun ne göreceğini **önce hesaplayın**, sonra deneyin.

Alıştırma 1.13 ★★☆ *Kod yaz ve raporla*

11. alıştırmadaki programı `-save-temps` ile iki adımda derleyin; Tablo 1.10'daki gibi .c, .i, .s, .o ve çalıştırılabilir dosya boyutlarını raporlayın. .i neden bu kadar büyük, .s'de hangi fonksiyon çağrılıyor?

Alıştırma 1.14 ★★★ *Tasarım (İnsan CPU)*

3 + 4 + 5'i hesaplayıp yazdıran oyuncak programın 8 hücreye sığıp sığmadığını gösterin. Sığmıyorsa, komut kümesinde **tek bir** değişiklikle (ör. OUT'un bellek yerine ACC'yi yazdırması) sığdırılabileceğini gösterip yeni programı yazın.

9. Kendini Değerlendirme

- ☐ Bilgisayarı soyutlama katmanlarıyla anlatabiliyor, ISA'nın neden bir “sözleşme” olduğunu açıklayabiliyorum.

- ☐ CPU'nun bileşenlerini (ALU, kontrol birimi, PC, IR) ve bellekteki adres–içerik ilişkisini açıklayabiliyorum.
- ☐ Oyuncak bir programı getir–çöz–yürüt döngüsüyle elle yürütebiliyor; 0–255 arasındaki sayıları onluk, ikili ve onaltılı tabanlar arasında çevirebiliyorum.
- ☐ Derleme hattının beş aşamasını sıralayabiliyor; GCC'yi kurup bir programı `-std=c17 -Wall -Wextra -pedantic` ile derleyip çalıştırabiliyorum.
- ☐ Bir hata iletisinin derleyiciden mi bağlayıcıdan mı geldiğini ayırt edebiliyorum; `echo $?` ile çıkış kodunu gözleyebiliyorum.

> **TAHMİN ET** · **Ön bilgi yoklamasına dönüş**
>
> **Soru 1:** RAM uçucudur, disk kalıcıdır; hızlı bellek pahalı ve uçucu, büyük bellek ucuz ve kalıcı olduğu için iki ayrı yer vardır. **Soru 2:** Her şey bit örüntüsüdür ('A' = 0x41, bir komut da bir sayıdır); anlamı onu okuyan program verir. **Soru 3:** İşlemci yalnızca makine kodunu yürütür; çeviriyi önişlemci, derleyici, çevirici ve bağlayıcı yapar, yükleyici sonucu belleğe yerleştirir.

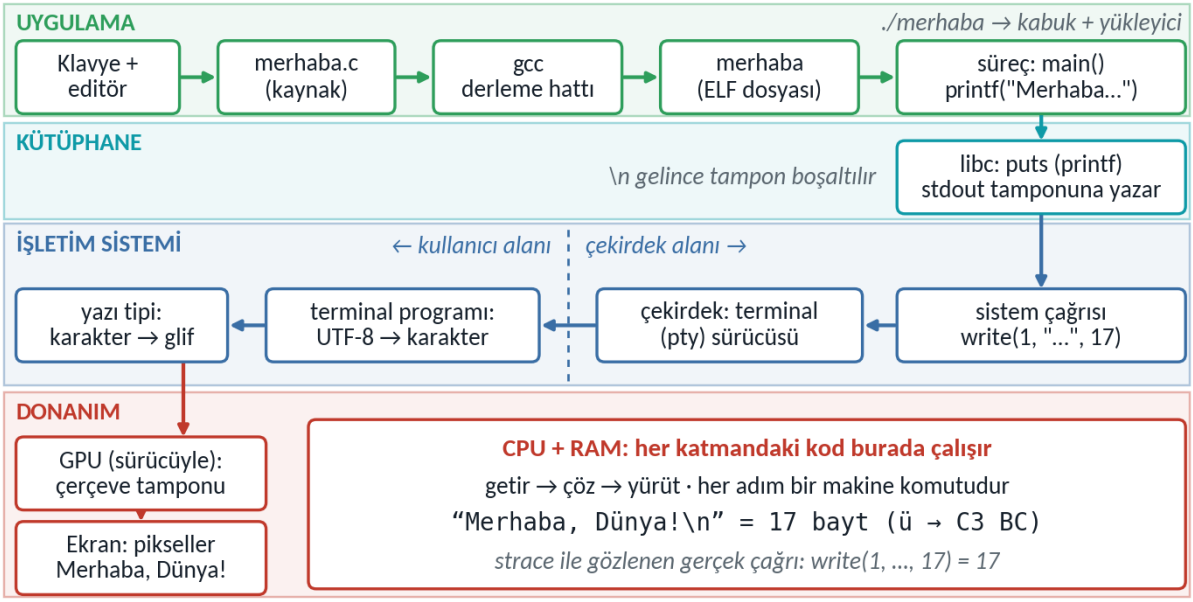
10. AP-2 Köprüsü ve Merak Kutusu

> **AP-2 KÖPRÜSÜ** · **Bellek hiyerarşisi ve algoritmaların gerçek hızı**
>
> AP-2'de algoritmaları önce **kaç adım** attıklarına göre karşılaştıracaksınız. Ama aynı sayıda adım atan iki algoritma gerçek bilgisayarda farklı hızda çalışabilir: Verisi önbellekte kalan program, sürekli RAM'e gidenden kat kat hızlıdır (Şekil 1.5'teki 4 saniye ile 5 dakika farkı). **Önbellek dostu** erişim, algoritmaların pratik performansını yorumlamanın temelidir. Haftanın ikinci tohumu şu sorudur: **"Aynı işi yapan iki program neden farklı hızda çalışır?"** İlk yanıtı Örnek 9'da gördük (eniyileme); dönem boyunca işlem sayma (H2, H6, H7), süre ölçümü (H7) ve önbellek deneyiyle (H11) bu soruya döneceğiz, AP-2'de ise biçimsel analizle yanıtlayacaksınız.

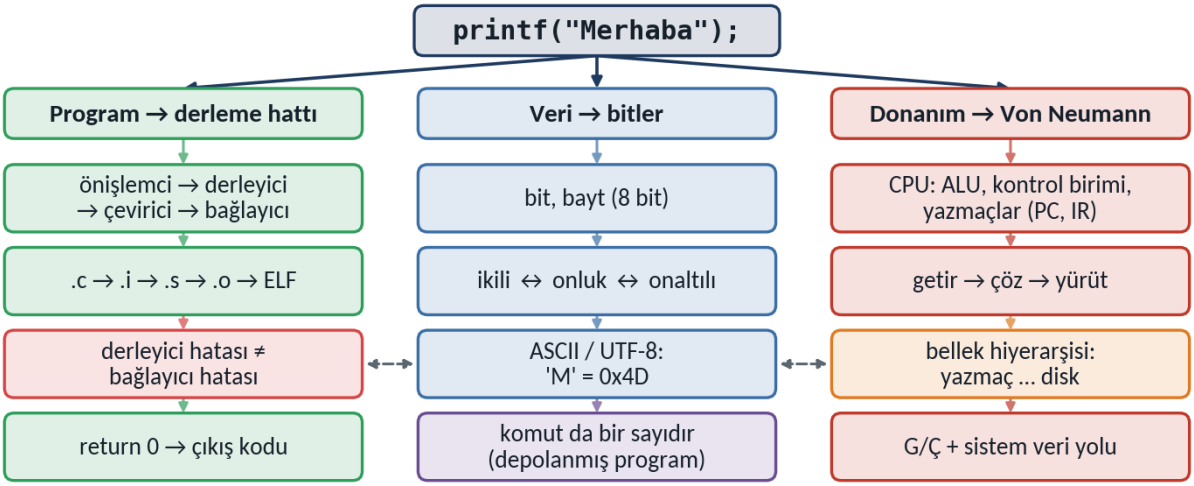
11. Özet, Kavram Haritası ve Terim Sözlüğü

Büyük Soru'nun yanıtı. `printf("Merhaba");` satırının klavyeden ekrana yolculuğu (Şekil 1.12, aşağıda): (1) Karakterler düzenleyiciyle `merhaba.c` dosyasına **UTF-8 baytları** olarak yazılır. (2) `gcc` **önişlemci → derleyici → çevirici → bağlayıcı** aşamalarını çalıştırır; `printf` çağrısı `puts`'a, o da makine koduna dönüşür ve libc'ye bağlanır. (3) `./merhaba` yazılınca **yükleyici** dosyayı RAM'e yerleştirir; CPU `main`'in komutlarını **getir–çöz–yürüt** döngüsüyle yürütür. (4) `puts` metni stdout tamponuna yazar; `\n` gelince çekirdeğe `write(1, …, 17)` **sistem çağrısı** yapılır. (5) Çekirdek baytları terminale iletir; terminal programı UTF-8 baytlarını karakterlere, yazı tipi gliflere çevirir; ekran kartı pikselleri çerçeve tamponuna yazar ve ekran onları yakar.

Şekil 1.12
"Merhaba, Dünya!"nın yolculuğu: kaynak koddan piksele sistem diyagramı



Şekil 1.13
Hafta 1 kavram haritası



Tablo 1.14
Hafta 1 terim sözlüğü (Türkçe–İngilizce)

Türkçe	İngilizce	Türkçe	İngilizce
soyutlama	abstraction	kaynak kod	source code
komut kümesi mimarisi	instruction set architecture (ISA)	önişlemci	preprocessor
merkezi işlem birimi	central processing unit (CPU)	derleyici	compiler
aritmetik mantık birimi	arithmetic logic unit (ALU)	çevirici	assembler
kontrol birimi	control unit	bağlayıcı	linker
yazmaç	register	yükleyici	loader
program sayacı	program counter (PC)	nesne dosyası	object file
getir–çöz–yürüt	fetch–decode–execute	tanımsız başvuru	undefined reference

Türkçe	İngilizce	Türkçe	İngilizce
bellek hiyerarşisi	memory hierarchy	kaçış dizisi	escape sequence
önbellek	cache	çıkış kodu	exit status
veri yolu	bus	sistem çağrısı	system call
ikili; onaltılı	binary; hexadecimal	hata ayıklama	debugging

Gelecek hafta: Algoritmik düşünme. İşlemci ne yapacağını kendisi bilmez; oyuncak bilgisayarımız bile 3 ile 4'ü ancak **adım adım, kesin ve sıralı** talimatlarla toplayabildi. Problemi bu kesinlikte adımlara ayırma becerisine **algoritmik düşünme** denir: H2'de IPO çözümlemesi, sözde kod, akış diyagramı ve izleme tablosuyla problemden algoritmaya gideceğiz; oyuncak bilgisayarda eksik olan "koşula göre sapma" ve "tekrar etme" yeteneklerinin neden vazgeçilmez olduğunu göreceğiz. **Hazırlık sorusu:** 1 + 2 + … + 100 toplamı bu haftaki komutlarla kaç komut tutardı? Gauss onu nasıl tek işlemle bulurdu? **Ek okuma:** Bryant ve O'Hallaron (2016, Bölüm 1), Patterson ve Hennessy (2017, Bölüm 1), Petzold (2022), Deitel ve Deitel (2022).

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Hopper, G. M. (1952). The education of a computer. In *Proceedings of the 1952 ACM National Meeting (Pittsburgh)* (pp. 243–249). ACM. https://doi.org/10.1145/609784.609818

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

National Museum of American History. (t.y.). *Log book with computer bug* (Nesne no. 1994.0191.01) [Müze nesnesi]. Smithsonian Institution. https://americanhistory.si.edu/collections/object/nmah_334663

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design RISC-V edition: The hardware/software interface*. Morgan Kaufmann.

Petzold, C. (2022). *Code: The hidden language of computer hardware and software* (2nd ed.). Microsoft Press.

Ritchie, D. M. (1993). The development of the C language. *ACM SIGPLAN Notices*, *28*(3), 201–208. https://doi.org/10.1145/155360.155580

von Neumann, J. (1945). *First draft of a report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania.